

A Prolog Typechecker

Abstract

Prolog is an untyped logic programming language developed in 1972 by Colmerauer and Kowalski. It maintains relevance today due not only to its applications in artificial intelligence and natural language processing, but also to its declarative nature that allows for a high-level approach to programming unique to logic programming languages. Its inherent untyped nature can result in runtime type errors that can be difficult to debug. This paper assesses past approaches to implementing a static type checking system in Prolog, and proposes a method to extend SWI-Prolog, a popular implementation of the language, to include such a system; the goal being to increase programming efficiency and safety in Prolog.

✓ I have not used any GenAI tools in preparing this assessment.

✓ I certify that all material in this dissertation which is not my own has been identified.

Signature: _____

1 Introduction

Programming languages come in many shapes, sizes and paradigms, and each has its own unique quirks. One such feature that varies widely between them is how, and indeed whether, they choose to manage type checking [1]. Java and C++ have static typing, explicitly declaring each variable with their corresponding permanent type [2], where in contrast Python has inferred dynamic typing, allowing variables to switch between types, and the compiler to take its best guess at how to treat the type [3]. Type systems can provide structure and reliability, reducing bugs and therefore increasing programmer efficiency [1]. Prolog, a logic programming language developed in 1972 by Colmerauer and Kowalski, is an untyped language [4], and therefore programming in it can be frustrating and time consuming.

Logic programming in general is a more modern style of programming, as stated by Korner et al. [5], “rather than measuring performance through the number of calculations per second, we can now do so through inferences per second”. They are declarative, allowing programmers to “state *what* is done while leaving the details of *how* it is to be done to the language implementation” [6]. They are, in general, inherently untyped [7], but this is not an explicit requirement of a logic programming language.

Korner et al. [5] discuss the “rich history” of Prolog, which has had many different implementations throughout the years. Different implementations vary widely in key features above and beyond the standard Prolog, and can even use different syntax, leaving them more akin to completely different languages. There have also been a number of languages created that are based on Prolog, for example, Mercury. Initially developed in the 1970s, it maintains relevance today by having applications in artificial intelligence and natural language processing. It is unique from other languages in this diversity, as new implementations are created to be tailored to the individual needs of research, compared to modern languages that have fewer implementations to be more commercially usable.

Defined in this same paper by Korner et al. [5], a Prolog implementation must at least support:

1. Horn clauses with variable terms and arbitrarily nested function symbols.
2. The ability to manipulate predicates and clauses as terms, aka the ability to write meta-predicates.
3. Selective Linear Definite clause resolution (SLD-resolution) as the basic execution mechanism.
4. Unification of arbitrary terms.
5. Automatic depth-first search of the tree for each logic query.

Prolog being an untyped language can present several problems when it comes to creating efficient and safe programs. A type system can increase the efficiency of programmers, and in a well-typed program, theoretically, things cannot “go wrong” at runtime [8]. Implementing a type system for Prolog could make programming in it significantly easier and safer.

2 Literature Review

This section of the paper will discuss previous approaches to creating a type system for Prolog, and look at SWI-Prolog, one of the most popular Prolog implementations currently [5].

2.1 Type System Designs for Prolog

There exist two original approaches to the problem of creating a type system for logic programs, on which many of the subsequent papers are based. Those two are the Hindley-Milner approach [9] and the Mycroft-O’Keefe approach [8].

Milner’s work [9] is the foundation behind type systems in both functional and logic programming languages, forming the basis for the type system behind languages like Haskell and Rust. It allows for type inference, meaning the compiler can infer undeclared types based on other declared types. This is a very simple but powerful type system, and still holds up today.

Mycroft and O’Keefe claim to be the first to write a paper specifically applying a polymorphic type system to Prolog [8]. They state that “a theorem prover which reasons about Prolog programs can be more powerful if it has type information available”. They further advocate for the inclusion of a type system by praising the time that can be gained by programmers during error checking. Their type system is polymorphic, allowing for the flexibility required to run Prolog programs, and runs static checks at compile time to ensure that they follow their slogan: “well-typed programs cannot go wrong”. It also uses type inference, avoiding explicit type declarations and making it less intrusive for the programmer but also more complicated to implement.

In addition, there are several newer logic programming languages that are very similar to Prolog, but that have added a type system amongst other features to extend functionality.

2.1.1 Prolog-like programming languages

Mercury is a logic programming language, built to improve upon Prolog whilst correcting its weaknesses. Somogyi, Henderson and Conway state that “many Prolog implementations perform no semantic checks at compile-time... the absence of declarations makes it difficult for compilers to gather the information required by the optimizations needed to achieve competitive performance” [6]. It has a very similar syntax to Prolog, but with the addition of type declarations. They advocate for the benefits of logic programming with type declarations - “once we got Mercury’s type, mode and determinism checks working, the number of bugs we had to chase ourselves, as opposed to the compiler pointing them out, dropped dramatically”. Its type system is based on Mycroft-O’Keefe, and is very similar to that of Gödel [10]; all types are checked at compile time, and every function, predicate and variable must be explicitly declared as a type. Mercury also has modes which specify whether an argument is an input or an output, which improves clarity and efficiency of the program as this data can be used by the compiler to generate more optimised code.

Lambda-Prolog (or λ -Prolog) is described as an extension of Prolog that includes polymorphic types as one of its additional features [11]. It uses higher-order hereditary Harrop formulas instead of traditional Horn clauses, but Nadathur and Miller [11] justify it as a true logic programming language as they state “the theory [permits] the logical connectives to be construed as certain simple search instructions while at the same time being true to their declarative intent”. Its type system allows for polymorphic types and requires explicit type declarations, as do most systems reviewed in this paper. It allows inherently for higher-order logic capabilities, which allows predicates to take other predicates or functions as arguments, but this does make it a much more complicated language than the much more practical Mercury or the focused on first-order logic original Prolog.

The developers of Gödel, Lloyd and Hill, state that “Prolog is not sufficiently expressive as the logic it uses is untyped” [12]. Their aim with Gödel was to improve the declarative semantics and create a more up-to-date programming language, improving upon Prolog’s foundation. It uses explicit type declarations, but allows for parametric polymorphism, and ensures all types are correct at compile time to avoid once again those run time pitfalls. It also includes type classes, letting predicates pick a class like “numeric” as its type rather than a specific narrow typing of “integer”, and it makes use of input/output modes in much the same manner as Mercury. However, its type system is much simpler than that of Mercury, allowing for an easier programmer learning curve.

2.1.2 Extensions of Hindley-Milner

Schrijvers, Costa, Wielemaker and Demoen take the modular approach [13], allowing users to use their system as a library on top of SWI or YAP Prolog and making the typing optional to allow for backwards compatibility. They emphasise this as a key advantage of their system, but it should also be noted that

despite this emphasis their system still is only compatible with two Prolog implementations, and not with other popular variants like XSB and GNU Prolog. They follow the Hindley-Milner type system, which they describe as “the most established type system”. They also use a “runtime type check” to check the boundary between typed and untyped code, which unfortunately does not fit the requirement of the Hindley-Milner system that programs cannot “go wrong” at runtime [9]. The modular system is a good one utility wise, as it doesn’t force programmers to learn a new implementation, but it does mean the type system is not necessarily a true one. They say “switching to a new language is usually not an option”, and address that by creating their system as a library add-on, allowing programmers to use their familiar Prolog implementation, as long as that is SWI or YAP. They make the type system optional to allow users to “gradually migrate” untyped code, and allow interfacing of typed and untyped code, so that it can be used in a modular way. Their type system syntax is very similar to that of Mercury.

2.1.3 Extensions of Mycroft-O’Keefe

Dietrich and Hagl [14] critique the translation of functional type systems into logic programs, claiming “these approaches to types in logic programming do not allow parametric polymorphism and the type information directly impacts the deduction mechanism which often results in a considerable restriction of the search space”. They take a “purely static” approach extended from Mycroft-O’Keefe, which allows them to stay truer to the way logic programs work. In addition, all checking takes place at compile time. This forces a programmer to fully type declare the program - therefore types are not dynamic and cannot change during the runtime - but it also means that unexpected type errors cannot occur after successful compilation. Dietrich and Hagl specifically highlight the weakness of Mycroft-O’Keefe’s system in that it does not account for subtyping, which can cause problems in Prolog, specifically due to its untyped nature. If a predicate is intended to take in an integer variable and output the same variable as a different type (for example, a float), it will be seen as ill-typed by Mycroft-O’Keefe, but this is built in as a possibility in Dietrich and Hagl’s extended version. To achieve this, they reject unification of type terms in favour of showing a set of subtype relations. This is a well laid out system in their paper, and creates an effective and functional type system. However, the definition of a Prolog implementation set out earlier in this paper necessitates the unification of arbitrary terms; by replacing this, the approach deviates far enough from the definition that it could be considered a new language, rather than an extended implementation of Prolog.

Lakshman and Reddy [7] again try to extend and add clarity to Mycroft and O’Keefe’s system. They try to approach the problem “prescriptively” - they explain this as the type system enhancing the language into a typed language, with typed semantics. They define a polymorphic type system similar to that of Lambda-Prolog, using specific type declarations as an integral part of the language to determine the programmer’s intention.

2.1.4 Other approaches

Pietrzak et al. [15] have yet another attempt at creating a new type system for Prolog, believing “this will have more practical impact than designing yet another strongly typed logic programming language”. They deviate from both Hindley-Milner and Mycroft-O’Keefe and describe their own parametrised type system, allowing a programmer to generalise type specifications for libraries or modules which can then be reused across programs. This sacrifices precision for efficiency, using only user or library defined types rather than allowing for any inference by the compiler, and minimizes reanalysis of modules. In reality, the loss of performance is minimal, and they claim in their paper that the tests they ran did show an improvement in efficiency of code analysis. However, it does add significant complexity for the programmer, forcing them to define their own type parameters. This can be complicated to learn, which does not quite align with the goal of type systems to make the programmer’s life easier.

2.2 SWI-Prolog

SWI-Prolog, like other Prolog systems, does not have a static type system built in [16]. It is however one of the most popular Prolog implementations currently [5], and it does support Constraint Logic Programming (CLP) and Constraint Handling Rules (CHR), which could theoretically act as a pseudo-type system. Neither is explicitly built for this purpose, with CHR designed for the modeling and implementation of constraint solvers, and CLP being made to solve pre-existing constraint domains [16]. Forcing these into a type checker would not be truly a good solution to the type system issue; both of these systems resolve only at runtime, and they also have a syntax that is not built for type checking, and therefore using them in that way creates code that can appear clunky and confusing.

As stated by Schrijvers et al. it is not always feasible for programmers to learn a new implementation in order to use a type system [13], and therefore extending SWI to have a true static type system would provide more utility.

3 Project Specification

Evidently, many attempts have been made to create a type system for Prolog, and the “correct” approach will be unique to each program and programmer. However, to try to preserve the essence of the language, this project will follow the static method set out by Mycroft and O’Keefe [8]. The implementation of type inference may be beyond the scope of this project, so to begin with a system will be created that type checks only variables that are explicitly typed by the programmer.

Explicit type-checking systems can act somewhat like documentation or code comments, making it easier for programmers other than the original creator to understand and edit a program without introducing new type errors. It will however produce more verbose code, which could potentially make the logic of the code harder to read - the intention is that the benefits of debugging efficiency and code clarity will outweigh this. This could also be mitigated by including some form of type inference in the system.

As mentioned previously, forcing a programmer to learn a brand new Prolog implementation is not efficient, and reduces some of that particular benefit of creating a type system. For that reason, this project intends to create a type system compatible with SWI-Prolog.

3.1 Aims and Objectives

This project aims to create a functional type system compatible with SWI-Prolog. It will be written in C, like SWI, and will have the following features:

- The type system will be **static** - variables will be assigned a type and remain that type throughout execution of the code.
- The type system will be **explicit** - types will be explicitly declared for each variable and parameter.
- The type system will check all types at **compile time** - a well-typed program in this system will not be able to go wrong at runtime.
- The type system will produce **clear error messages** - the programmer will be able to see exactly what is incorrectly typed and where.

Due to the time frame of this project, it is possible that not all features of a Mycroft-O’Keefe system will be feasible to implement. Therefore, the following features will be stretch goals to be attempted once a working core system has been established:

- The type system could use **parametric polymorphism** - parameters can be declared as, for example, type T, and T can be declared elsewhere.

- The type system could use **type inference** - programmers can choose not to declare some types, and the compiler will infer types based on the variable's interactions with other typed variables.

3.2 Schedule of Work

An agile methodology will be used to complete this project, splitting the work into six sprints, two weeks long each. This method allows for a continuous workflow of implementation and verification, letting the project evolve as more functionality is implemented so that the stretch goals can be easily incorporated if necessary. Agile also lets the development remain focused on the end product.

3.2.1 Sprint 1 - Type Annotation Parser

The goal of the first sprint is to modify the parser to recognise type annotations. This will require designing a basic syntax for type declarations with a keyword trigger so that the parser can identify which lines should be read as type declarations.

Deliverables for this sprint:

- A simple type annotation syntax.
- The ability to run Prolog code with type annotations added, even if the type annotations do not function at this stage.

3.2.2 Sprint 2 - Basic Compile-Time Checks

This sprint will aim to implement the basics of the type checker to see if variables passed into predicates match the types that the predicates require for basic types, for example *integer* and *atom*. This will require associating the type declarations with predicates.

Deliverables for this sprint:

- Errors are thrown for basic type errors.

3.2.3 Sprint 3 - Error Reports

During this sprint, detailed error reports will be created, aiming to highlight which type and which line of code is the one that is incorrect. This sprint will also be an opportunity to continue working on any aspects of previous sprints that have not been completed, and to reassess the content of the future sprints if necessary.

Deliverables for this sprint:

- The predicate that caused the type error is identifiable from a clear error message.
- A revised plan for the future sprint content.

3.2.4 Sprint 4 - Composite Types and Documentation

This sprint will aim to expand the type system to handle composite types like *list(Type)*. It will also be focused on creating the necessary documentation and working on the project report; to begin working on documentation before the foundations of the code are in place would result in large amounts of rewriting if and when features are changed in accordance with the agile methodology.

Deliverables for this sprint:

- Functional recursive type-checking of composite types.
- Clearly commented code.
- An initial draft of the development section of the project report, including an explanation of what has been implemented so far.

3.2.5 Sprint 5 - Stretch Goals or Bug Fixing

This sprint is reserved to allow the project the extra time it needs for unfinished features due to slippage, or to work on the stretch goals laid out previously. Documentation will also be continued during this sprint.

Potential deliverables for this sprint, to be altered to allow for slippage or stretch goals:

- An initial draft of the evaluation section of the project report.
- A final draft of the earlier sections of the project report.

3.2.6 Sprint 6 - Final Touches and Documentation

This sprint is to fix any final small bugs, and to finalise the project report.

Potential deliverables for this sprint, to be altered to allow for slippage or stretch goals:

- The completed final project report.
- The completed type system code, that achieves all the aims set out for this project in the previous section.

After each sprint the progress made will be assessed, and the plan for the next sprint will be adjusted accordingly. Particularly for the later sprints, the work set out to be done has been left intentionally flexible to allow the project to develop in an agile way.

3.3 Evaluation Methods

This project will be successful if it fulfils Mycroft and O’Keefe’s slogan of “well-typed programs cannot go wrong”; the program will not fail during runtime with a type error. The system should correctly identify at compile time all type errors, and provide a clear explanation to the programmer of where and what each error is.

To evaluate how well the type system works, it will be tested on dummy Prolog programs with a range of different type errors deliberately built in. The number of correctly identified errors and any missed errors or false positives will be measured as a way to enumerate the project’s success. The clarity of each of the error messages will also be assessed, noting whether the correct predicate or line of code has been identified as the one with the type error.

3.4 Ethics and Risks

There will be no human participants or human data used in this project, and there are no specific ethical concerns to be considered. The purpose of this project is merely to enhance an existing programming language for increased programmer efficiency and safer programs.

This project is a relatively safe one, but when using open-source code created by other developers as this project requires, there is always the potential risk of errors or a poor maintenance update on the source code of SWI-Prolog causing issues with further development. The project could also be affected if the SWI developers decide to remove their code from GitHub. This risk can be mitigated by storing an unchanged original source file locally. All project code will also be backed up into cloud storage to guard against the risk of data loss.

4 Conclusion

This paper has analysed a variety of approaches to creating a type checking system for Prolog. It identifies a chosen approach based on Mycroft and O’Keefe’s proposed system, and proposes a method and agile schedule for its implementation. The project intends to increase both ease and efficiency of programming in Prolog by providing a clear and foolproof method of correcting type errors.

References

- [1] B. C. Pierce, *Types and programming languages*. MIT press, 2002.
- [2] —, *Advanced topics in types and programming languages*. MIT press, 2024.
- [3] A. J. Dhruv, R. Patel, and N. Doshi, “Python: the most advanced programming language for computer science applications,” *Science and Technology Publications, Lda*, pp. 292–299, 2021.
- [4] P. M. Nugues, *An introduction to prolog*. Springer, 2006.
- [5] P. Körner, M. Leuschel, J. Barbosa, V. S. Costa, V. Dahl, M. V. Hermenegildo, J. F. Morales, J. Wielemaker, D. Diaz, S. Abreu *et al.*, “Fifty years of prolog and beyond,” *Theory and Practice of Logic Programming*, vol. 22, no. 6, pp. 776–858, 2022.
- [6] Z. Somogyi, F. J. Henderson, and T. C. Conway, “Mercury, an efficient purely declarative logic programming language,” *Australian Computer Science Communications*, vol. 17, pp. 499–512, 1995.
- [7] T. Lakshman and U. S. Reddy, “Typed prolog: A semantic reconstruction of the mycroft-o’keefe type system.” in *ISLP*, vol. 91, 1991, pp. 202–217.
- [8] A. Mycroft and R. A. O’Keefe, “A polymorphic type system for prolog,” *Artificial intelligence*, vol. 23, no. 3, pp. 295–307, 1984.
- [9] R. Milner, “A theory of type polymorphism in programming,” *Journal of computer and system sciences*, vol. 17, no. 3, pp. 348–375, 1978.
- [10] D. Jeffery, F. Henderson, and Z. Somogyi, “Type classes in mercury,” in *Proceedings 23rd Australasian Computer Science Conference. ACSC 2000 (Cat. No. PR00518)*. IEEE, 2000, pp. 128–135.
- [11] G. Nadathur and D. Miller, *An Overview of Lambda-PROLOG*. Citeseer, 1988.
- [12] P. Hill and J. W. Lloyd, *The Gödel programming language*. MIT press, 1994.
- [13] T. Schrijvers, V. Santos Costa, J. Wielemaker, and B. Demoen, “Towards typed prolog,” in *Logic Programming: 24th International Conference, ICLP 2008 Udine, Italy, December 9-13 2008 Proceedings 24*. Springer, 2008, pp. 693–697.
- [14] R. Dietrich and F. Hagl, “A polymorphic type system with subtypes for prolog,” in *European Symposium on Programming*. Springer, 1988, pp. 79–93.
- [15] P. S. Pietrzak, J. C. Fernández, G. Puebla, and M. V. Hermenegildo, “A practical type analysis for verification of modular prolog programs,” in *Proceedings of the 2008 ACM SIGPLAN symposium on Partial evaluation and semantics-based program manipulation*, 2008, pp. 61–70.
- [16] J. Wielemaker, T. Schrijvers, M. Triska, and T. Lager, “Swi-prolog,” *Theory and Practice of Logic Programming*, vol. 12, no. 1-2, pp. 67–96, 2012.